PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 12

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum Implementasi ISR Otomatis

1. Tambahkan revalidate

- Modifikasi static.tsx pada folder src/pages/produk

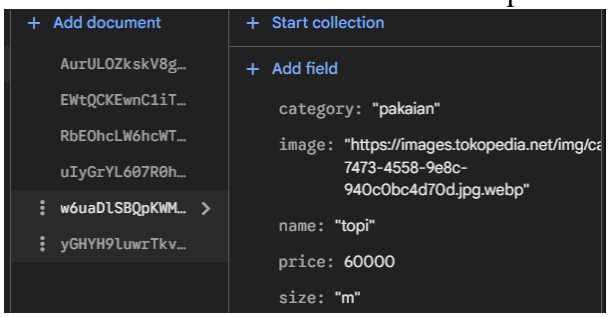
```
my-app > src > pages > produk > static.tsx > getStaticProps
import TampilanProduk from "../../view/produk";
import { ProductType } from "../../type/product.type";

const halamanProdukStatic = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Static</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukStatic;

export async function getStaticProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response: { data: ProductType[] } = await res.json();

    return {
        props: {
            products: response.data,
        },
        revalidate: 10, // Revalidate the data every 10 seconds
    };
}
```

2. Pengujian ISR

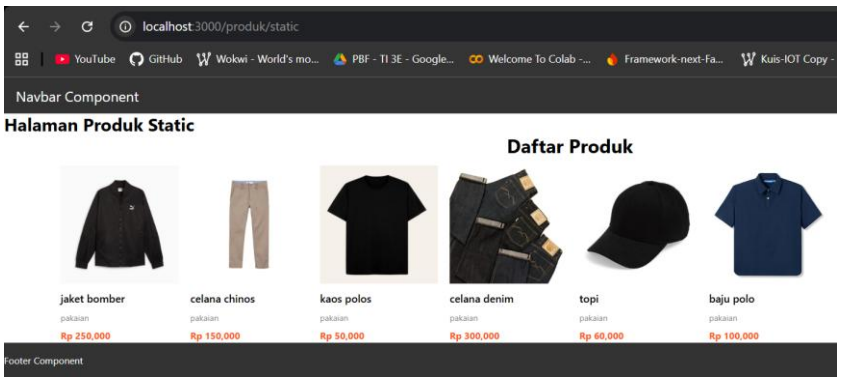
- Jalankan: (lakukan hal sama seperti JS sebelumnya untuk ngebuild SSG)

```
PS E:\PBF_2026\JS_12\my-app> npm run build
 ○ /about
 ƒ /api/[[...produk]]
 ƒ /api/hello
 ○ /auth/login
 ○ /auth/register
 ○ /blog/[slug]
 ○ /category/[...slug]
 ○ /produk
 ○ /produk/[id]
 ○ /produk/index_duplicate
 ƒ /produk/server
 ƒ /produk/server/[id]
 ● /produk/static (500 ms)
 ● /produk/static/[id] (2444 ms)
   /produk/static/AurULOZkskV8gs3tjVCK (493 ms)
   /produk/static/EWtQCKEwnC1iTBNcU0ID (490 ms)
   /produk/static/RbEOhcLW6hcWT8B3oBg1 (488 ms)
   /produk/static/uIyGrYL607R0hXaIjspR (487 ms)
   /produk/static/yGHYH9luwrTkv0bi3KJy (486 ms)
 ○ /profile
 ○ /profile/edit
 ○ /setting/app
 ○ /shop/[[...slug]]
 ○ /user
 ○ /user/password
```

- Tambahkan data baru di database pada firebase



- Refresh setelah 10 detik



Praktikum On-Demand Revalidation

1. Buat API Revalidate

- Buat file revalidate.ts pada folder pages/api/ dan modifikasi

```
my-app > src > pages > api > revalidate.ts > handler
import type { NextApiRequest, NextApiResponse } from 'next';

type Data = {
    revalidated: boolean;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    try {
        await res.revalidate('/produk/static');
        return res.json({ revalidated: true });
    } catch (error) {
        console.error('Error revalidating:', error);
        res.status(500).json({ revalidated: false });
    }
}
```

2. Tambahkan Parameter Data

- Modifikasi file revalidate.ts

```
import type { NextApiRequest, NextApiResponse } from 'next';

type Data = {
    revalidated: boolean;
    message?: string;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    if (req.query.data === "produk") {
        try {
            await res.revalidate('/produk/static');
            return res.json({ revalidated: true });
        } catch (error) {
            console.error('Error revalidating:', error);
            res.status(500).json({ revalidated: false });
        }
    }

    return res.json({
        revalidated: false,
        message: "Invalid query parameter, Expected 'data=produk'.",
    });
}
```

- Uji menggunakan parameter yang sesuai saat akses /revalidate

```
localhost:3000/api/revalidate?data=produk
YouTube  GitHub  Wokwi - World's mo...  PBF - T
{
    "revalidated": true
}
```

- Uji dengan mengosongkan parameter data saat akses /revalidate

```
localhost:3000/api/revalidate?data=
YouTube  GitHub  Wokwi - World's mo...
{
    "revalidated": false,
    "message": "Invalid query parameter, Expected 'data=produk'."
}
```

3. Tambahkan Token Security

- Buka file .env dan modifikasi

```
REVALIDATE_TOKEN = 12345678
```

- Modifikasi file revalidate.ts

```
my-app > src > pages > api > TS revalidate.ts > handler
import type { NextApiRequest, NextApiResponse } from 'next';

type Data = {
    revalidated: boolean;
    message?: string;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    if (req.query.token !== process.env.REVALIDATE_TOKEN) {
        return res.status(401).json({
            revalidated: false,
            message: "Insert correct token",
        });
    }
    if (req.query.data === 'produk') {
        try {
            await res.revalidate('/produk/static');
            return res.json({ revalidated: true });
        } catch (error) {
            console.error('Error revalidating:', error);
            res.status(500).json({ revalidated: false });
        }
    }

    return res.json({
        revalidated: false,
        message: "Invalid query parameter, Expected 'data=produk'.",
    });
}
```

- Uji jika token benar

```
localhost:3000/api/revalidate?data=produk&token=12345678
YouTube  GitHub  Wokwi - World's mo...  PBF - TI 3E - Google...
{
    "revalidated": true
}
```

- Uji jika token salah

```
localhost:3000/api/revalidate?data=produk&token=123
YouTube  GitHub  Wokwi - World's mo...  PBF - TI 3E - Google...
{
    "revalidated": false,
    "message": "Insert correct token"
}
```

Tugas Praktikum

1. Tambahkan lagi produk pada firebase

```
+ Add document                    + Start collection
  EWtQCKEwnC1iTBNcU0ID            + Add field
  RbEOhcLW6hcWT8B3oBg1              category: "pakaian"
  VQSk1bXt06gjgUS0cm1l              image: "https://www.footlocker.id/media/catalog/product/V/A/VAS000D3HY28-2.jpg"
  uIyGrYL607R0hXaIjspR              name: "sepatu"
                                    price: 400000
                                    size: "44"
```

2. Implementasikan ISR dengan revalidate: 10.

my-app > src > pages > produk > static.tsx > getStaticProps

```
import TampilanProduk from "../../view/produk";
import { ProductType } from "../../type/product.type";

const halamanProdukStatic = (props: { products: ProductType[] }) => {
    const { products } = props;
    return (
        <div>
            <h1>Halaman Produk Static</h1>
            <TampilanProduk products={products} />
        </div>
    );
};
export default halamanProdukStatic;

export async function getStaticProps() {
    // Simulate an API call to fetch products
    const res = await fetch('http://localhost:3000/api/produk');
    const response: { data: ProductType[] } = await res.json();

    return {
        props: {
            products: response.data,
        },
        revalidate: 10, // Revalidate the data every 10 seconds
    };
}
```

3. Tambahkan endpoint On-Demand Revalidation.

```
import type { NextApiRequest, NextApiResponse } from 'next';

type Data = {
    revalidated: boolean;
    message?: string;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    if (req.query.data === "produk") {
        try {
            await res.revalidate('/produk/static');
            return res.json({ revalidated: true });
        } catch (error) {
            console.error('Error revalidating:', error);
            res.status(500).json({ revalidated: false });
        }
    }

    return res.json({
        revalidated: false,
        message: "Invalid query parameter, Expected 'data=produk'.",
    });
}
```

4. Tambahkan validasi token.

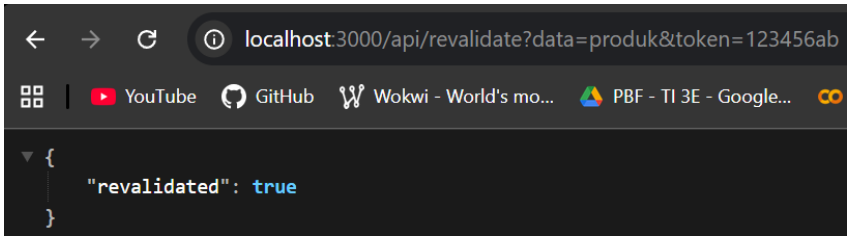
```
REVALIDATE_TOKEN = 123456ab
```

my-app > src > pages > api > revalidate.ts > handler

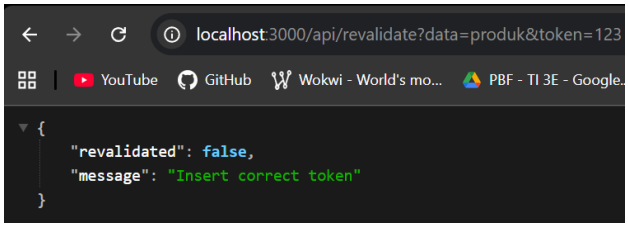
```
import type { NextApiRequest, NextApiResponse } from 'next';

type Data = {
    revalidated: boolean;
    message?: string;
};

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    if (req.query.token !== process.env.REVALIDATE_TOKEN) {
        return res.status(401).json({
            revalidated: false,
            message: "Insert correct token",
        });
    }
    if (req.query.data === 'produk') {
        try {
            await res.revalidate('/produk/static');
            return res.json({ revalidated: true });
        } catch (error) {
            console.error('Error revalidating:', error);
            res.status(500).json({ revalidated: false });
        }
    }

    return res.json({
        revalidated: false,
        message: "Invalid query parameter, Expected 'data=produk'.",
    });
}
```

5. Uji dengan:

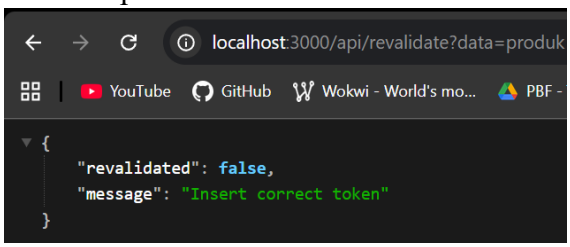
- Token benar



- Token salah



- Tanpa token



Pertanyaan Refleksi

1. Mengapa ISR lebih fleksibel dibanding SSG?

Jawab: ISR dapat memperbarui konten secara otomatis tanpa harus menjalankan ulang proses build secara menyeluruh, sedangkan SSG mengharuskan eksekusi manual npm run build setiap kali ada perubahan data agar konten terbaru tampil di halaman

2. Apa perbedaan revalidate waktu dan on-demand?

Jawab: Revalidate waktu memperbarui halaman secara otomatis di balik layar berdasarkan interval waktu (dalam detik) yang sudah dikonfigurasi di getStaticProps, sedangkan on-demand memicu pembaruan secara manual melalui pemanggilan endpoint API tertentu sesuai kebutuhan

3. Mengapa endpoint revalidation harus diamankan?

Jawab: Endpoint perlu dilindungi agar pihak yang tidak berwenang tidak bisa sembarangan memicu proses regenerasi halaman hanya dengan mengakses URL tersebut

4. Apa risiko jika token tidak digunakan?

Jawab: anpa token, siapa saja bisa mengakses URL revalidate dan memaksa server melakukan pembaruan data, yang berpotensi merusak konsistensi tampilan maupun membebani performa server

5. Kapan ISR lebih cocok dibanding SSR?

Jawab: ISR lebih sesuai untuk konten yang tidak terlalu sering berubah dan mengutamakan performa, karena halaman tetap disajikan dari cache statis. Sebaliknya, SSR lebih ideal ketika

data berubah sangat cepat karena setiap permintaan pengguna selalu menghasilkan render terbaru langsung dari server